

SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY —
INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

**Balanced SRAM Repair: A Meta-Heuristic
Approach to Optimizing Good Die per
Wafer**

Fraj Yassine Lakhali

SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY —
INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

**Balanced SRAM Repair: A Meta-Heuristic
Approach to Optimizing Good Die per
Wafer**

**Ausgewogene SRAM-Redundanz: Ein
Metaheuristischer Ansatz**

Author: Fraj Yassine Lakhel
Supervisor: Prof. Ulf Schlichtmann
Advisor: Felix Last, Susanne Henkel, Stefan Drexel
Submission Date: 15-05-2023

I confirm that this bachelor's thesis is my own work and I have documented all sources and material used.

Munich, 15-05-2023

Fraj Yassine Lakhal

Acknowledgments

I want to sincerely thank my advisors, Felix Last, Susanne Henkel and Stefan Drexler, for all the insights that were of immense help for my thesis. I want to also thank them for putting the time and effort to come up with a suitable research topic.

I want to thank my family for their support and for their encouragement to pursue my academic journey.

Abstract

Implementing memory repair on SoCs is an industry standard solution in order to achieve higher product yield and which results in an increased good die per wafer. Different repair schemes result in varying product yield depending on process technology, screening profile and product design, including factors like on-die recovery of full blocks and systems. The target of this paper is to optimize the resulting good die per wafer, by balancing the area overhead of memory repair against its yield benefit. By looking into the real die dimensions, die x/y and on-die logic recovery, we achieve a more accurate optimization of good die per wafer and hence improved repair assignment for the SoC leading to less product cost.

Contents

Acknowledgments	iv
Abstract	v
1 Introduction	1
2 Theoretical Background	2
2.1 Yield	2
2.2 Defects	3
2.3 Repair	3
2.3.1 Challenges of Adding Repair	4
2.4 Optimization	4
2.5 Search Methodologies	5
2.5.1 Search Space	5
2.5.2 Optimum Solutions	5
2.5.3 Metaheuristics	5
2.5.4 Constraints	6
2.6 Evolutionary Algorithms	6
2.7 Genetic Algorithms	6
2.7.1 Initialization	7
2.7.2 Evaluation	7
2.7.3 Selection	7
2.7.4 Recombination	8
2.7.5 Mutation	8
2.7.6 Replacement	8
2.7.7 Convergence	8
3 Related Work	9
4 Proposed Approach	10
4.1 Models	10
4.1.1 Gross Die per Wafer Model	10
4.1.2 Yield Model	11

4.1.3	Area Estimation	13
4.2	Approaches	13
4.2.1	Using a Repair Efficiency Threshold	14
4.2.2	Metaheuristic optimization	16
4.2.3	General Remarks	17
5	Results	18
5.1	Baselines	18
5.1.1	No Repair	18
5.1.2	Randomly Assigned Repair	18
5.2	Experimental Setup	18
5.3	Methods Comparison	19
5.4	Investigating Yield and Area Overhead	19
5.4.1	Effect of Die Size	21
5.4.2	Effect of Memory Content	22
6	Conclusion	24
	List of Figures	25
	Bibliography	26

1 Introduction

A die or a chip contains logic, memory content and other circuitry. Embedded memories are part of the fastest accessible memories and are typically used for caching. A typical chip contains as many as 5000 of such memories and contribute up to 60% of the total chip size. This means the yield or in other terms the percentage of usable bits of those memories are critical. This metric can be observed during manufacturing and is determined by a combination of different factors. Past studies revealed that memory redundancy and the way it is laid out on the chip highly impacts area overhead and yield. Through redundancy, different types of memory failures can be resolved. By using the available memory redundancy patterns, the impact of the fails is reduced with the goal of increasing Good Dies per Wafer (GDPW). Furthermore, varying parameters like die size and memory content potentially influence this metric. The deviations in these parameters and the importance of their influence will be part of our experiments. This paper is organized as follows. In Section II, the theoretical foundation is laid, including a summary of the state of the art yield estimation and computational optimization. Section IV, we present algorithms that find a memory redundancy layout that achieves the best GDPW result. In Section IV, we do a comparison of the different methods used. We also conduct studies of the influence of changes in the input parameters on the GDPW. In the last section, we draw a conclusion based on the findings.

2 Theoretical Background

In this chapter, we explain the concepts that form the basis of this work and define important terms. We first discuss semiconductor yield and memory redundancy, before introducing relevant aspects of metaheuristic optimization and genetic algorithms.

2.1 Yield

Yield can be defined as the percentage of non-defective bits out of the total bits. The yield of a chip is composed by a number of different yield components:

$$\text{Yield per unit area } Y_F = \frac{\text{quantity of layers discharged}}{\text{quantity of layers introduced into the production process}}$$

$$\text{Evaluation Yield } Y_B = \frac{\text{quantity of chips passing the disc test}}{\text{quantity of chips delivered for the disc test}}$$

$$\text{Assembly Yield } Y_M = \frac{\text{number of chips mounted in package without defects}}{\text{number of chips delivered for assembly}}$$

$$\text{Test Yield } Y_P = \frac{\text{number of functional chips}}{\text{Number of chips delivered for final test}}$$

The resulting yield [Wid+00] is therefore:

$$Y = Y_F Y_B Y_M Y_P$$

Due to their aggressive design rules, embedded memories are more susceptible to manufacturing defects and field reliability issues compared to other embedded cores within a System on Chip (SoC). As a result, they are often referred to as "defect magnets". As memory bit density increases, the overall yield of the SoC becomes more dependent on the memory yield. Therefore, achieving a high memory yield is crucial to achieving cost-effective silicon. [ZS03]

Memory compilers are widely used in modern integrated circuit (IC) design flows to generate memories. These tools are typically offered by the manufacturing foundry

or third-party vendors and provide a library of parametrizable memories that are verified and characterized for the specific technology node. By specifying parameters such as the number of words, word width, and other architectural parameters, a memory compiler can provide the necessary results for both the IC design flow and manufacturing. [LHS20]

A typical product today may include over one thousand embedded memory instances of various bit cell and compiler types. Technology nodes continue advancing making the number of memories on the chip rise significantly. Optimizing GDPW and enhancing the yield is crucial to increase cost-effectiveness and volume output.

2.2 Defects

Vmin is the minimum operating voltage of a memory [GC21]. A die on a wafer is considered bad if it contains at least one defect that causes it to malfunction, whether it is a hard defect or a Vmin-related fail. Hard defects, such as shorts or opens, can occur at any voltage supply. Soft fails, also known as Vmin fails, occur only at a specific voltage level for memories. The defects that occur on memory are categorized into various types. These defects include single bit (SB) fails and wordline (WL) defects. The occurrence and proportion of these fail types depend on the bit cell type and density.

Foundries typically measure random hard defects in terms of the number of defects per square inch or cm, which is referred to as defect density D_0 . Standard logic is a part of the circuitry that serve as an interface for the memory. Because memories and memory arrays are designed with high density, their failure probability is can be twice or triple as high as that of standard logic.

2.3 Repair

To enhance wafer yield and lower the cost of testing per good die, redundancy is utilized to repair defects that can potentially be fixed. Accurately estimating the number of dies gained after applying redundancy is crucial in predicting the volume of a product. Memory providers provide the option of adding repair elements to the selected memory, which can replace the defective portion of the memory with a spare element and address any defects that may arise on a die. [Ram]

Sense amplifiers are part of the critical peripheral circuits of an SRAM. Their role is to ensure a faster access to data by sensing and amplifying the small signals on the bitline [TH15]. We improve the data transfer of memories by using column-multiplexing or mux. It works by accessing several bits in a single cycle [Cha+01]. There are various repair schemes available that address different defect mechanisms, based on

the technology. When applied, a complete IO (muxed bitlines) for example is added to the memory array to replace one complete IO containing a defect, additionally mux and sense amplifier logic can be repaired.

Combining repair schemes is a common approach to improve the efficiency of repairing defects. However, we limit our experiments to IO Repair.

2.3.1 Challenges of Adding Repair

In general, incorporating memory repair schemes into an SoC design can come with several costs and challenges. One of the primary costs is the area overhead associated with implementing the repair structures. This can increase the overall die size, which can affect the yield and ultimately the cost of the product.

Performance of the memory can be affected by increasing access time or power consumption. This can affect the overall system performance, particularly in high-performing SRAMs where speed is critical. [Mer+19]

Using repair also requires additional integration effort, such as modifying the design flow to include repair insertion and testing, and the development of specialized repair algorithms. This can add complexity to the design process. [Mer+19]

Finally, incorporating repair mechanisms also requires additional testing requirements, such as testing the repaired memory and verifying the functionality of the repair structures. This can increase the overall testing time and cost, as well as the complexity of the testing process.

Therefore, while memory repair can improve the yield and reduce product cost, it is important to carefully balance the benefits against the associated costs and challenges to ensure that the overall product meets the desired specifications and requirements.

The repair assignment poses a challenge of accurately estimating the yield loss, selecting the appropriate repair schemes, and computing their corresponding overheads. This is done with the aim of maximizing the number of good dies per wafer by balancing the benefits of yield improvement against the repair costs. Ultimately, the objective is to achieve an optimal balance specific to each product that will result in the maximum number of good dies per wafer.

2.4 Optimization

The term "optimization" refers to the process in which we attempt to find the best possible solution amongst all those available. Optimization requires an evaluation function which assigns a quality of a given solution and a search algorithm that minimizes or maximizes that objective function. [14] An optimization problem consists of a set of variables $X = \{x_1, \dots, x_n\}$, a set of domains $D = \{D_1, \dots, D_n\}$, a set of

constraints $C = \{f_1, \dots, f_m\}$, and an objective function $f_0(X)$. A solution of the problem is an assignment of a value to every variable in X such that every constraint in C is satisfied. [Liu+14]

2.5 Search Methodologies

Search methodologies operate within a search space to retrieve the optimal solution.

2.5.1 Search Space

The problem space refers to the entire range of solutions that could potentially appear in a given problem. Searching through a vast problem space is a routine approach, whether the objective is to identify a feasible solution (to satisfy a constraint of a problem), the optimal solution (in an optimization problem), or an answer to a specific question (when a decision need to be made). [Liu+14] As the problem space increases in size, so does the set of potential solutions. These solutions can either be generated before the search by enumerating them (if feasible) or can be generated during runtime while performing the search. [Liu+14] Multiple challenges arise from the "variable/feature dimension", "function nonlinearity", "structural heterogeneity", etc... which can hinder the search efficiency and effectiveness are often hindered. [Liu+14] This complexity can obscure the solution space and make it difficult to enumerate potential solutions either prior to the search or during runtime. Additionally, in some problems, the solution space is continuous, rendering enumeration impractical. [Liu+14] The complexity of the search space, which is determined by factors such as the domain of variables, the number of variables, and the structure of constraints, can significantly affect the efficiency and effectiveness of a search. [Liu+14] A good search strategy works nonetheless by taking the shortest path to the optimum solution from a set of different paths. [Liu+14]

2.5.2 Optimum Solutions

Optimum solutions are defined within an interval relative to the total search space. Local optimum is a point in a subset of that search space which is better than all other points. Global optimum is the best solution we can find in the total interval.

2.5.3 Metaheuristics

The term "meta-heuristic" encompasses the use of heuristics that are guided by a meta strategy, meaning a high-level set of methods, or even define some available moves to perform that changes a solution into another one. The algorithm is guided by an

evaluation rule [14]. The search algorithm is an algorithm that navigates the defined search. One way of searching the search space without requiring prior knowledge of the fitness landscape is to apply meta-heuristics. We opt for a solution that is beyond local.

2.5.4 Constraints

To perform a meta-heuristic search for an optimum solution, there are usually constraints or conditions. We are focusing on a set of constraints called hard constraints. These conditions render our solution vector feasible only if each condition is met. Suppose we take a repair assignment problem for one memory as an example. In this case, a hard constraint could be the requirement that the memory can have a positive amount of repair. In the event of a violation of a hard constraint, it renders the solution infeasible [14].

2.6 Evolutionary Algorithms

We are looking into a subset of the meta-heuristic approaches, namely evolutionary methods. The name comes from the fact that they follow the natural process of evolution. They maintain a population of possible solutions over a number of generations and the survival is awarded to the fittest of the individuals making up the population.[14]

2.7 Genetic Algorithms

Genetic algorithms (GAs) are search algorithms that follow the principles of natural selection and genetics. They rely on a fitness measure to determine an individual's relative fitness. This serves as a guide to the evolution of good solutions. [14]

Being a subset of evolutionary algorithms, GAs work with a population of candidate solutions. The size of the population is specified by the user. Scalability and performance of GAs depend to an extent on the choice of the population. A smaller population causes our algorithm to converge before we achieve an unsatisfying solution whereas a big population will waste computing resources. [14]

The chosen population will go through a several iterations until we converge to an optimal solution. After we encode the problem and define a fitness measure, we can start the evolution of solutions. The solution will be compared against each other in terms of fitness. The 7 different steps to each genetic algorithm are next to be defined as seen on the figure Figure 2.1.

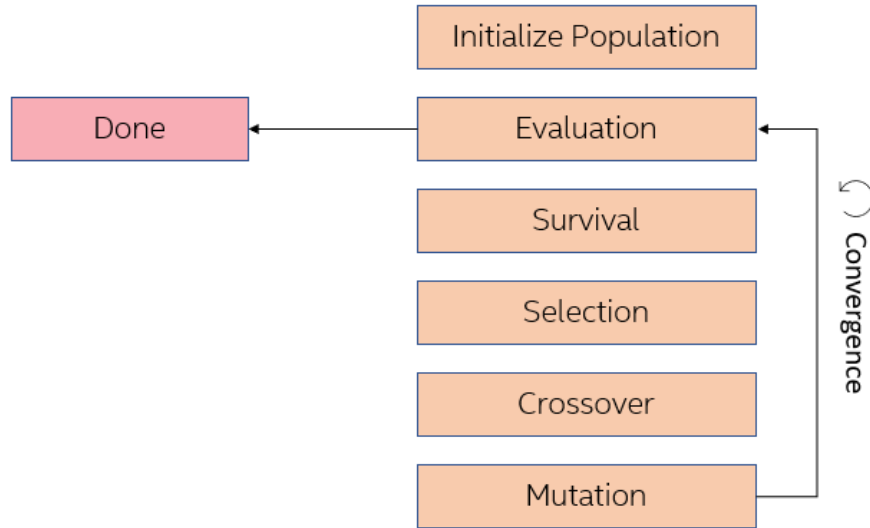


Figure 2.1: Genetic Algorithm Steps

2.7.1 Initialization

The algorithm starts off with an initial population of candidates that are either randomly generated or curated by the by the domain-specific experts. The latter can be easily implemented in the first generation. [14]

2.7.2 Evaluation

Evaluation refers to the step in which we calculate the fitness of our population. The population can either be the initial one or an offspring. [14]

2.7.3 Selection

Selection involves assigning more copies to solutions with higher fitness values. We leverage the survival-of-the-fittest mechanism in the population of candidate solutions. The primary objective of selection is to prioritize superior solutions over inferior ones, and various selection procedures have been proposed for achieving this objective. Some selection procedures include roulette-wheel selection, stochastic universal selection, ranking selection, and tournament selection. [14]

2.7.4 Recombination

in genetic algorithms, recombination refers to the process of combining bits and pieces of two or more parent solutions to create new offspring solutions that may be better than their parents. Achieving successful recombination performance depends on the proper design of the mechanism used, which can be achieved through various methods. It is important to note that the offspring generated through recombination will not be identical to any individual parent, but will instead inherit novel combinations of traits from both parents. [Gol02]

2.7.5 Mutation

The process of mutation involves introducing random and local modifications to an individual solution, in contrast to recombination which operates on multiple parents. Although various forms of mutation exist, they generally involve making one or more changes to an individual's traits. This means that mutation can be seen as a random walk in the vicinity of a candidate solution. [14]

2.7.6 Replacement

After the selection, recombination, and mutation phases, a new population of offspring is generated and replaces the original population. Various replacement techniques, including elitist replacement, generation-wise replacement, and steady-state replacement methods, are used in genetic algorithms to determine how the new population replaces the old one. [14]

2.7.7 Convergence

Steps 1-6 are then repeated in the last step until stopping criteria are met. The most frequently utilized stopping criterias may include a pre-determined maximum iteration limit, having a specific solution quality level, or no change in the best solution after a certain number of iterations. These criteria may be employed alone or in conjunction with each other. [14]

3 Related Work

GDPW optimization has been the topic of many published scientific papers. There has been some progress done by many authors, [Ng+20] and [deV05] to name some examples, that discuss ways of modelling, estimating, and optimizing good dies per wafer, however they don't consider memory redundancy. Other authors for example like in references [Seg+99] and [RTB02], overlook the GDPW optimization and focus solely on achieving the best repair assignment. Reference [Seg+99] is using a rule-based algorithm. It extrapolates the yield results based on a 256 KiB block of memory depending to the size of the memory in question. Reference [RTB02] however does the repair assignments through a statistical analysis of electrical bitmaps from manufactured wafers. This is done by adding row or column repair increments and calculating yield improvement based on the probabilities already accumulated for each fail bitmap signature. Finally, the authors of Reference [Mel06] successfully apply GDPW optimization using memory redundancy. The iterative approach is based on cycling through the memories one by one (potentially many full memory list cycles are done) where each time an optimal repair scheme is applied. The optimization of the single memory is relative to the memories already optimized and the other chip components. This method is computationally expensive as mentioned in the paper: 10 options for 10 memories will result in 1010 combinations. As a result, the method is limited regarding the number of memories to optimize.

4 Proposed Approach

4.1 Models

4.1.1 Gross Die per Wafer Model

Gross die per wafer refers to the total number of dies produced per wafer. We are using a counting approach instead of an estimation to ensure more accurate results. The iterative counting approach works as follows: We fit in as many dies as possible to form a first line on the upper edge of the wafer. We repeat this iteratively to form a row each time with dies across the wafer.

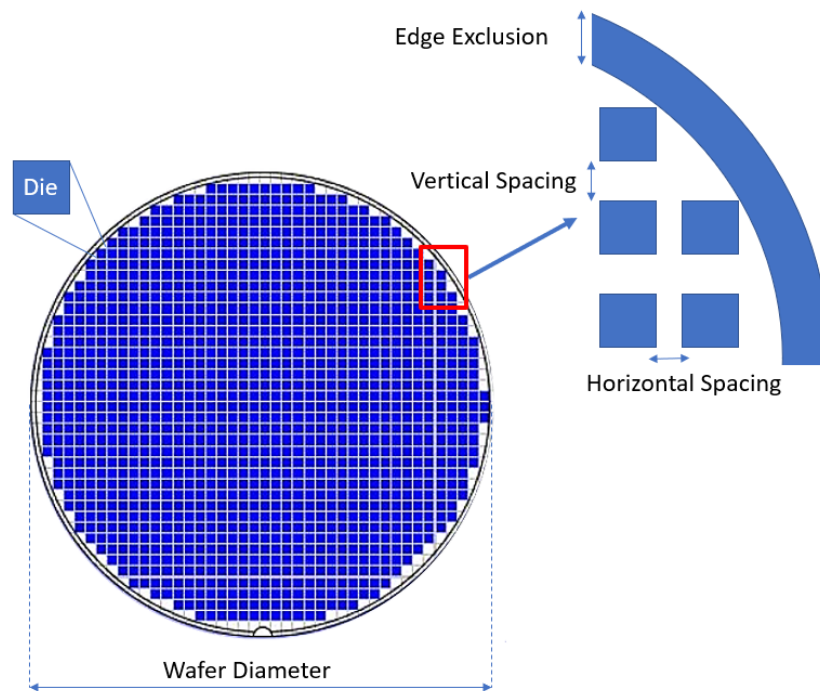


Figure 4.1: Dies Printed on a Wafer

For our assumptions on how the wafer manufacturing criteria, we need to define

edge exclusion, vertical and horizontal spacing. Edge exclusion is the area following the edge of the wafer that makes printed dies invalid due to significant yield loss. Horizontal spacing is the horizontal space between adjacent dies along the y-axis and vertical refers to the space on the x-axis. For this calculation we assume we have a die size of 10 mm² plus the area estimation of our memories, a wafer diameter of 300 mm², an edge exclusion of 3 mm, a horizontal space of 0.16 mm and a vertical space of 0.35 mm. We assume the total area of the memories we are optimizing to be separate from the die size to simplify the calculations. It is irrelevant later on in the comparison of our different experiments as this an assumption done across all experiments. With all these assumptions while also assuming that a die size of 10 squared mm, we get a gross die per wafer of 5666 dies.

4.1.2 Yield Model

To achieve accurate yield modelling, the understanding of process conditions (defect density, fail pareto, bitcell detail, bitcell Vmin condition), design information (product information, memory inventory, mission profile (use condition of product, target market)), layout of the die (die size) and test data (screening (testing) conditions such as temperature and voltage) is essential. A comprehensive yield model considers any potential defective die that we assume is part of the yield loss. We must understand the production flow. The same die is printed several times on the wafer, as a result we can map back the yield to the die if we divide the total GDPW by the gross die per wafer.

We assume that random defects are modelled, no process systematic, design related defects occur or that these can be fixed to achieve the best possible yield.

The core yield model results from two yield components, the modelling of hard defect yield and of Vmin yield. The product of these two yield components describes the core yield model:

$$Y_C = Y_{BDD} Y_{V_{min}}$$

Literature reports a variety of models that are discussed for estimating the impact of hard defects. The differences in these models arise from their underlying assumptions about the distribution of the defect density D_0 . One established model for silicon manufacturing is the Price model, which assumes that the defect density follows an exponential distribution. The yield can be calculated using this model by:

$$Y_{DD,P} = \frac{1}{(1 + AD_0)^n}$$

[Pri70]

Where A is the total die area of the design and n is the process complexity factor which reflects the number of critical layers. Not all of the layers on the wafer are critical to the yield and we consider the ones that affect it.

An improvement over the Price model is the Bose-Einstein-Model, which considers the critical die area and includes a design factor k that accounts for the varying defect sensitivity of different chip areas A_i . The yield calculation in this model is based on the defect density limitation.

$$Y_{BDD} = \frac{1}{(1 + kAD_0)^{n'}}$$

where

$$k = \frac{\sum k_i A_i}{A}, \text{ and } A = \sum A_i$$

The calculation of yield for embedded memories also involves the consideration of Vmin fails, which is the second most significant component of the core model for random defects. Unlike hard defects, Vmin fails occur at low voltage operation and are electrically observable in embedded memories.

Let's consider the yield of a single die consisting of logic and one memory. The yield can be thus modeled as follows:

$$Y_G = Y_{logic} Y_{memory} = Y_{logic} Y_{CCA} Y_{periphery}$$

The Bose-Einstein model is utilized to determine the yield of the standard logic chip area.

The yield of the memory periphery is denoted as $Y_{periphery}$ and comprises standard logic components that follow the Bose-Einstein-Model, similar to Y_{logic} . The third and final component, Y_{CCA} , represents the yield of the core cell array.

$$Y_{logic} = \frac{1}{(1 + k_{logic} A_{logic} D_0)^n}$$

$$Y_{memory} = Y_{periphery} \cdot Y_{CCA} = \frac{1}{(1 + k_{logic} A_{periphery} D_0)^n} \cdot Y_{CCA}$$

The yield of the unrepaired core cell array memory is determined by two core yield components as previously described. The core cell array yield, denoted as Y_{CCA} , is simply the inverse value of the core cell array fail rate, represented by F_{CCA} :

$$Y_{CCA} = 1 - F_{CCA}$$

The fail rate refers to the likelihood of a defect causing a failure in the core cell array, and is used to determine the value of F_{CCA} for the unrepaired memory. The fail pareto, which provides information on the types and occurrences of defects, is necessary.

$$F_{CCA} = F_{CCA}\delta_{SB} + F_{CCA}\delta_{VTB} + F_{CCA}\delta_{HTB} + F_{CCA}\delta_{WL} + F_{CCA}\delta_{BL} + \dots$$

Where

$$\delta_{SB} + \delta_{VTB} + \delta_{HTB} + \delta_{WL} + \delta_{BL} + \delta_{DQ} + \delta_{QB} + \delta_{others} = 1$$

These $\delta_{failtype}$ probabilities are taken from the fail paretos.

The repair model breaks down each component of the sum and calculates them individually. An example of this is shown below to illustrate the probability of a core cell array failure $F_{SB} = F_{CCA}\delta_{SB}$ caused by a single bit fail, represented by F_{SB} . F_{SB} is the probability that out of N possible bits in the core cell array, one bit fails.

$$\begin{aligned} F_{SB} &= 1 - Y_{SB} \\ Y_{SB} &= (1 - \delta_{SB})^N \\ \delta_{SB} &= 1 - (Y_{SB})^{\frac{1}{N}} = 1 - (1 - F_{SB})^{\frac{1}{N}} \end{aligned}$$

The fail rate for all other fail types are equivalently calculated. To enhance the simple model of one memory on a die, the following applies to an unrepaired die as fails occur independently:

$$Y_G = Y_{logic} \cdot \prod Y_{mem,i}$$

Having defined our model consecutively for unrepaired yield and repaired yield, we then simply multiply the yield and the gross die per wafer to get the GDPW of the respective die.

4.1.3 Area Estimation

We are using a machine learning model [LHS20] to estimate area based on the compiler of the memory (contains information about the bit cell size...) and the total number of bits or the size of the bit cell and whether we include the repair columns/rows/words if they are in use. Area estimation is relevant for estimating the gross die per wafer.

4.2 Approaches

The approaches that we present follow the same pattern shown in 4.2

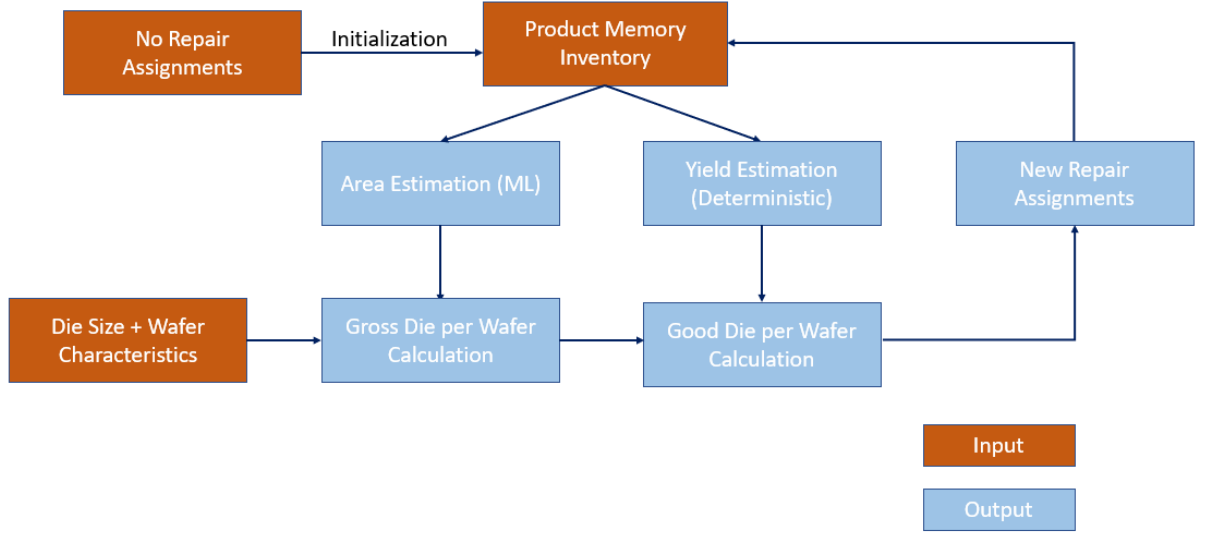


Figure 4.2: Workflow of the Suggested Approaches

4.2.1 Using a Repair Efficiency Threshold

we use the efficiency threshold to control the amount repair assigned to each instance. This method has two parts: a first part that optimizes repair according to a chosen threshold. A second part that optimizes the threshold while searching for the best GDPW.

Optimize Repair for a Given Threshold

The algorithm is given an efficiency threshold which is our target efficiency for each instance. All instances start off with an efficiency metric calculated for a no repair scenario, meaning no repair is applied to any of the instances.

We then iterate through all possible repair schemes, amount and combinations; we calculate the efficiency of the instance and then compare it to the old efficiency. We define the efficiency as follows:

$$\text{Eff} = \frac{Y_{\text{Benefit}}}{A_{\text{Penalty}}}$$

We define yield benefit and area penalty as follows:

$$Y_{\text{Benefit}} = \frac{Y_{\text{PostRepair}}}{Y_{\text{Native}}}$$

where post repair yield $Y_{PostRepair}$ is the resulting yield of adding repair to the respective memory and the native yield Y_{Native} is the yield of the memory without any repair.

$$A_{Penalty} = \frac{A_{TotalChip} + A_{Overhead}}{A_{TotalChip}}$$

where area overhead $A_{Overhead}$ is the area of the added circuitry from the additional repair and $A_{TotalChip}$ being equivalent to the die size.

If our new efficiency is better than the old, calculated efficiency or exceeds one plus the efficiency threshold we set, we replace the respective efficiency threshold with the new value.

Based on the definition of repair efficiency, more repair is applied when a smaller efficiency threshold is used and bigger in the other case. This comes down to the fact that repair for a smaller instance results in a big area penalty in contrast to the yield benefit thus not worth repairing. Let's pick a small memory with an area of 743.133 with a word depth of 12 and word width of 64. Applying one IO repair will result in an area penalty of 1.00001033 and a yield benefit of 1.000000345 (we use rates and not percentages). Therefore, our efficiency is around 0.99999001. If we don't apply any repair, the values are 1.0 for all of the three metrics. Picking a threshold on the bigger side of the spectrum like 1.01 our chosen memory will indeed not be repaired. If we pick a smaller efficiency around 1.00001, we see that the memory fits the criteria to be repaired.

By employing this method, we achieve a more granular repair assignment that avoids cases where small memories being repaired causes a relatively significant overhead and we target solely the biggest memories.

Optimize the Thresholds for a Best Possible GDPW

The best efficiency threshold in terms of GDPW is found using a binary search.

The binary search works as follows: we set the smallest and the biggest threshold of our search interval, we compute the GDPW for the two boundaries of the interval, if we find that the best GDPW is achieved through the smallest threshold we pick the lower half of our initial interval so our new smallest threshold is our old one our biggest threshold is now the middle value of the old interval. similarly, we pick the upper half of the interval in the case that the best GDPW is from the biggest threshold. If the two values are equal, we do the following: we save the best threshold found so far and reduce the width of the interval by 0.00001 on each side.

For each threshold, as stated earlier, we must compute the GDPW. This will be our criteria for going to the next iteration. However, we must set the number of iterations at the beginning and the smallest and biggest threshold of our interval. It is unclear

how many iterations are needed to find the best GDPW nor where the best threshold will be.

4.2.2 Metaheuristic optimization

Genetic Algorithm

The last modeling problem is metaheuristic optimization. We try to find the best repair assignment for each instance based on the resulting GDPW. We do this by calculating the GDPW in the objective function we are optimizing. Our approach is based on a genetic algorithm implementation from a python library called "pymoo". For our experiment, we are using the basic $(\mu + \lambda)$ genetic algorithm for single-objective problems. This algorithm is a type of evolutionary algorithms that has a μ number of parents/individuals selected for breeding and λ is the number of offspring. This variant of algorithms keeps a population constant across all generations. This objective function which is in our case the GDPW calculation which is subject to some constraints. The constraints have to do with the amount of repair allowed for each compiler for each instance. This amount varies according to the compiler type, design package and technology of the memories list.

Problem Definition

We must adapt our original problem of GDPW optimization to fit into the framework of a metaheuristic algorithm.

The objective function will interpret our population each time it is run and gives out the GDPW results. It will take the lists of repair lists and will return a list of GDPW values.

We refrained from defining the constraints separately. We instead opted for defining the constraints as part of the objective function. This is called constraint violation as objective. We compute how far off our repair list is from feasibility by adding up the differences of each repair value from the feasible repair value. We assigned a relatively big positive value to infeasible solutions.

In the case of feasible solutions, we would get a negative value. This is due to the fact that all repair amounts are within the feasibility interval. We avoid calculating the objective function for the infeasible solutions by only calculating the GDPW for the repair lists if they have a negative constraint violation value. This way we avoid the downside of the constraint violation of subjective method, that in the other case will have to do many redundant evaluations of our objective function.

However, we could not avoid the fact that we you must call the evaluate function many times(at least in the first generations) because we get a lot of infeasible solutions.

On the other hand, our constraint function is orders of magnitude less costly than the objective function. The trade off is acceptable in our experiment.

Algorithm Definition

Defining the algorithm is a matter of choosing the algorithm and tuning its parameters to fit our use case.

As we are working with genetic algorithms, we have to specify first the population size and the sample population. We are running experiments with population sizes of 12, 15 and 20 over 150, 125 and 100 generations respectively. As a first population, we are passing a list of individuals including one with maximum repair options for each instance (in our experiment, we limit this to IO repair) and one with the minimum (meaning no repair). The rest of the individuals are randomly generated followed the same idea as in the method we showcased earlier with the random repair assignments. This will come to our advantage as we start off already with some feasible solutions to help our algorithm converge faster to the optimum solution vector.

Next, we are defining the crossover and the mutation. For our experiment, we are using simulated binary crossover, short term SBX. We chose the probability of mutation to be equal to one, this means that all the genes of the parents will be modified in the offspring. We are also using polynomial mutation, short form PM. The distribution index of which defines the shape of the probability density function used in generating the offspring solutions is set to 0.3. Please note that the probability distribution of polynomial distribution is the same as simulated binary crossover.

For both crossover and mutation, we are enabling rounding repair. This is because our problem is discrete. The repair amount in each repair type cannot be a floating number because we can only add full columns of redundancy.

Minimize Function

Instead of minimizing our objective, we need to maximize for GDPW. we simply negate the objective function.

4.2.3 General Remarks

In all our experiments that involve repair, we calculated the yield of unrepairable memories separately one time at the beginning. This is an optimization step to avoid calculating GDPW for unrepairable memories for each new repair list. We simply must multiply the yield and gross die per wafer of unrepairable memories for unrepairable memories with the yield of repairable memories and their gross die per wafer.

5 Results

In this section, we discuss the experimental setup and results. We compare the results of each method. We finally conduct an investigation on GDPW and how it is influenced by the chip characteristics in terms of size and memory content,

5.1 Baselines

5.1.1 No Repair

The first method we employ does not involve any repair of the memory instances. This means we calculate GDPW without factoring in any additional repair area overhead. We are using this method as our baseline. This method has the lowest GDPW.

5.1.2 Randomly Assigned Repair

In the second method, we adopt a random repair assignment for each instance. We specifically go each instance by instance in the search for the minimum and maximum repair option available. This ensures that the repair chosen is indeed supported by the memory compiler and can be implemented on a real die. We do this for each instance and each repair type and then concatenate the list into a list of repair assignments that is to calculate the post repair yield and the resulting gross die per wafer with the area overhead created by the instances that receive repair.

5.2 Experimental Setup

We use 603 memories which represent a subset of memories from a real product. Our sample memories include 15 unrepairable memories (ROMs) and 588 repairable memories. All repairable memories allow for a single IO repair, or no repair. The die size is set to 50 mm^2 and the memory content is set to 36% of the total die area. Two methods (no repair and threshold) are deterministic, whereas two are stochastic (random assignment and metaheuristic optimization). To evaluate the variability of the two methods, we calculate the error values using the standard error of the mean (SEM), which is a measure of the variability of the sample mean. For the random assignment

and metaheuristic methods, we ran the simulation 10 times to obtain 10 results, and calculated the SEM for each set of results. The SEM was calculated by dividing the sample standard deviation by the square root of the sample size.

5.3 Methods Comparison

We compare the results of our methods based on the resulting GDPW. Figure 5.1 shows the results of each method. With this setup the gross die per wafer is 875. No repair yields 770.24 GDPW. The threshold method outperformed all of the other methods. Being deterministic, the threshold algorithm has a consistent GDPW of 826.1, followed by the metaheuristic approach with a value of 823 in all three repetitions. Random assignment has around 810.21 good dies per wafer on average.

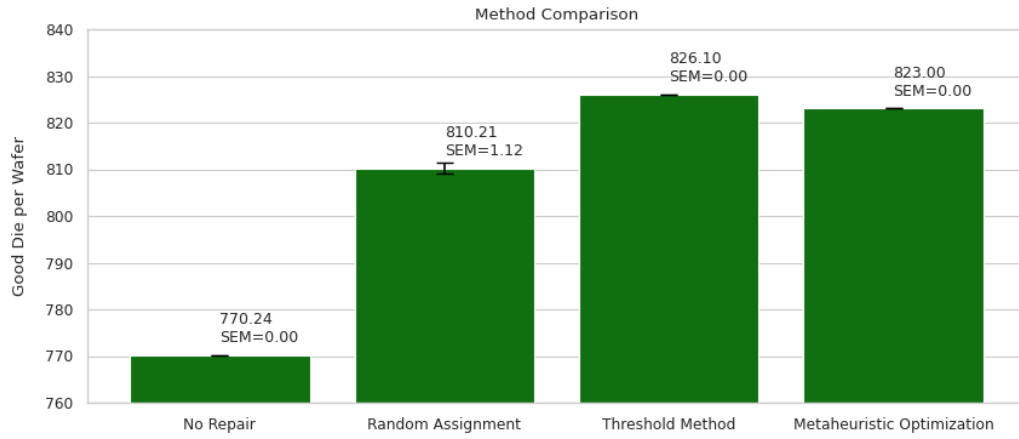


Figure 5.1: Methods Comparison

The metaheuristic approach gives consistent results with the fitness converging to 823 in all of the runs. On figure 5.2, we can see an example of an optimization run.

5.4 Investigating Yield and Area Overhead

In this section, we want to see the indirect effects of the die size and memory content on the optimization of the GDPW. After changing a parameter (die size or memory content), we re-optimize all memories and record the resulting GDPW. For the matter, we are going to use the threshold method, as it is the best performing approach.

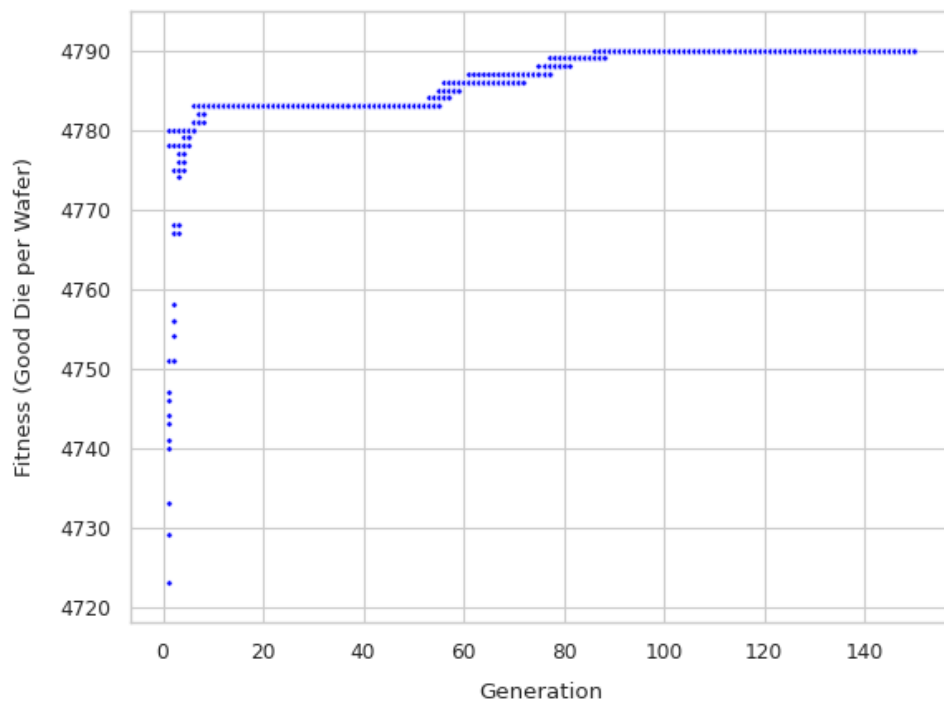


Figure 5.2: Genetic Algorithm's GDPW over Generations

5.4.1 Effect of Die Size

When changing the die size incrementally by adding 10mm² and keeping the memory content constant at 18% of the total die size, we see the yield increase to reach a maximum of 99.43%. The yield values recorded in the experiments are shown in 5.3. The increase is however not linear and rather a zig-zag. A possible explanation is that not enough iterations are made and before the the yield originating from to the best GDPW is reached, the optimization is stopped.

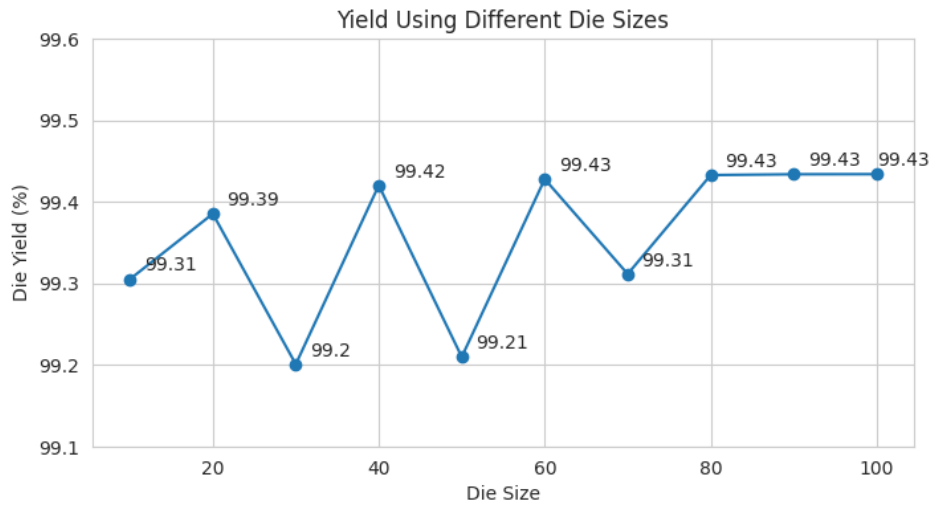


Figure 5.3: Yield using Different Die Size

We see a similar trend when it comes to area overhead. Shown in figure 5.4, we kept the x-axis with the different die sizes. We refer to the original area overhead as the area overhead that we get from the 10mm² die size. For the y-axis we calculated the percentage of the new area overhead in relation to the original overhead. The area overhead increases to reach a maximum of 102.52% relative to the original area overhead.

This can be explained by the fact that higher yield comes at the cost of area, so the higher the yield the higher the area overhead. Like in figure 5.3, we observe the same effect.

Yield increase comes at the expense of area. The prioritization of the yield over the area increase could be explained by the lower significance of the memories area increase relative the total die. This makes the cost from the added circuitry bearable and not contributing to a noticeable level on the overall GDPW.

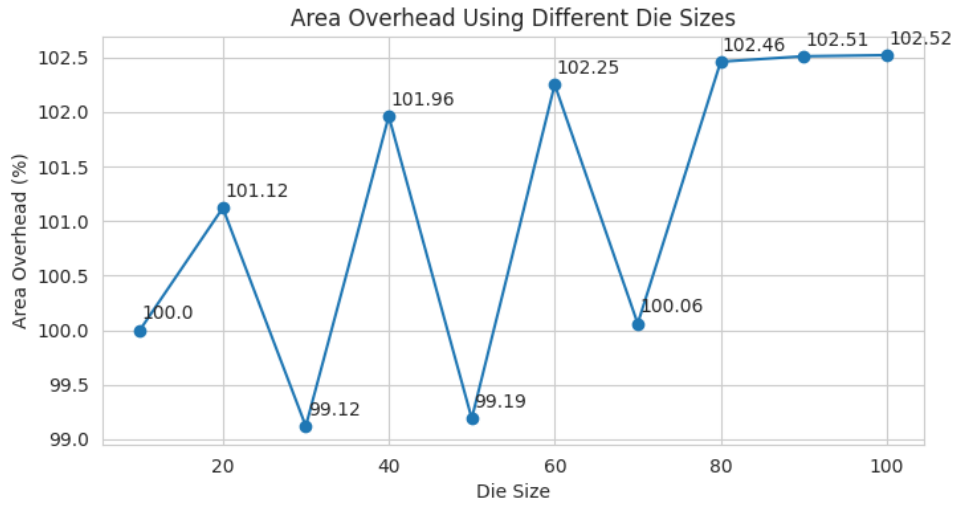


Figure 5.4: Area Overhead Using Different Die Size

5.4.2 Effect of Memory Content

For our next investigation, we change the memory content incrementally from 18% to 72% and we keep the die size constant at 10mm². The yield function of the memory content in figure 5.5 is linearly decreasing from a maximum of 99.305% to a minimum of 97.589%.

In figure 5.6, we take a different approach than earlier. As the memory content increases, we will necessarily see a higher area overhead. This does not showcase the effect of the different memory contents on the area overhead. We therefore calculated the area overhead per megabyte of memory content in figure 5.6. We see from the plot that the area overhead per megabyte decrease when we have more memory content.

Increasing the memory content seems to have a negative effect on yield and a positive one on the area increase. When we optimize the GDPW, we see that the area is prioritized. More memory content means less significance of the yield of the total memories on the whole chip. The yield change does not impact the GDPW as much compared to the area overhead. That's why we also see a decrease in the relative area overhead.

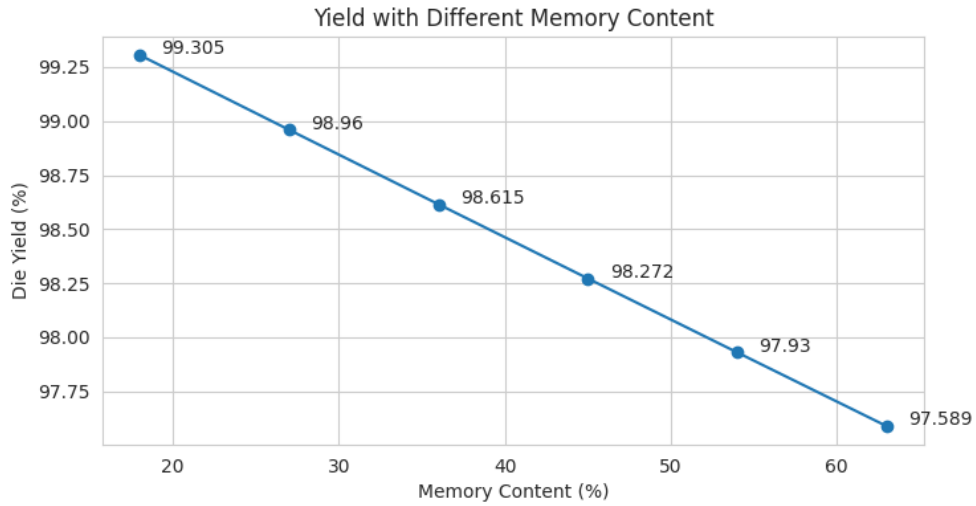


Figure 5.5: Area Overhead Using Different Memory Content

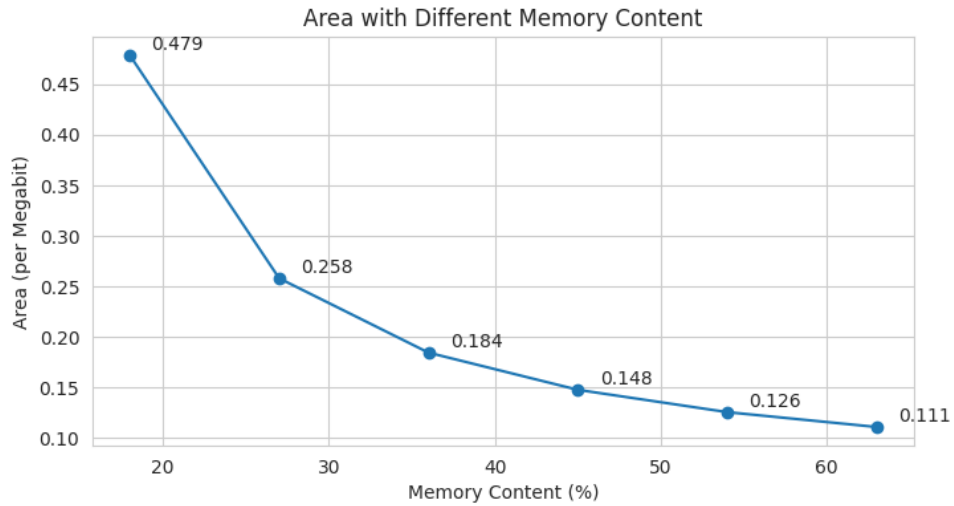


Figure 5.6: Area per Megabyte using Different Memory Content

6 Conclusion

In this work, we set out to optimize the GDPW by balancing repair onto the embedded memories of the die.

We looked into two approaches. The first approach used a binary search algorithm to find the best efficiency threshold metric that results in the best GDPW. The second method introduced an application of metaheuristic optimization that maximizes the GDPW.

Results showed the the threshold method had the best GDPW output followed by the optimization's method. The metaheuristic approach, being a black box approach to the GDPW optimization problem, could not outperform a more tuned approach, namely the threshold method. The threshold method treats repair simultaneously as a singular and a combined problem of repair assignment with the goal of achieving best GDPW. Repair is assigned only if the efficiency exceeds the input efficiency threshold making repair more granular and at the same time we search for the best overall GDPW. The combination of the two leads to better results. This is not the case for metaheuristic optimization that treats all memories as a single unit (individual in the genetic algorithm's terms) and computes the GDPW directly.

As we investigated the effect of yield and area, we found a positive effect on yield when increasing die size . In contrast, a negative effect was found with an increase of the memory content.

It is worth noting that we only used one repair scheme in our evaluation. Using additional repair schemes is an open question that could be explored in future work. It is possible that a different repair scheme may yield better results for a specific technology node, screening profile, or product design. Additionally, combining multiple repair schemes may further improve the GDPW, but this would require careful consideration of the area overhead, performance impact, integration effort, and testing requirements of each scheme. Further research in this area could provide valuable insights into optimizing GDPW.

List of Figures

2.1	Genetic Algorithm Steps	7
4.1	Dies Printed on a Wafer	10
4.2	Workflow of the Suggested Approaches	14
5.1	Methods Comparison	19
5.2	Genetic Algorithm's GDPW over Generations	20
5.3	Yield using Different Die Size	21
5.4	Area Overhead Using Different Die Size	22
5.5	Area Overhead Using Different Memory Content	23
5.6	Area per Megabyte using Different Memory Content	23

Bibliography

- [14] *Search Methodologies: Introductory Tutorials in Optimization and Decision Support Techniques*. en. Boston, MA: Springer US, 2014. ISBN: 978-1-4614-6939-1. DOI: 10.1007/978-1-4614-6940-7.
- [Cha+01] K. Chakraborty, S. Kulkarni, M. Bhattacharya, P. Mazumder, and A. Gupta. "A physical design tool for built-in self-repairable RAMs." In: *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 9.2 (Apr. 2001), pp. 352–364. ISSN: 1063-8210, 1557-9999. DOI: 10.1109/92.924057.
- [deV05] D. deVries. "Investigation of Gross Die Per Wafer Formulas." en. In: *IEEE Transactions on Semiconductor Manufacturing* 18.1 (Feb. 2005), pp. 136–139. ISSN: 0894-6507. DOI: 10.1109/TSM.2004.836656.
- [GC21] S. Gupta and B. H. Calhoun. "Dynamic Read V MIN and Yield Estimation for Nanoscale SRAMs." In: *IEEE Transactions on Circuits and Systems I: Regular Papers* 68.3 (Mar. 2021), pp. 1171–1182. ISSN: 1549-8328, 1558-0806. DOI: 10.1109/TCSI.2020.3044836.
- [Gol02] D. E. Goldberg. *The Design of Innovation*. Vol. 7. Genetic Algorithms and Evolutionary Computation. Boston, MA: Springer US, 2002. ISBN: 978-1-4757-3645-8. DOI: 10.1007/978-1-4757-3643-4.
- [LHS20] F. Last, M. Haeberlein, and U. Schlichtmann. "Predicting Memory Compiler Performance Outputs Using Feed-forward Neural Networks." en. In: *ACM Transactions on Design Automation of Electronic Systems* 25.5 (Sept. 2020), pp. 1–19. ISSN: 1084-4309, 1557-7309. DOI: 10.1145/3385262.
- [Liu+14] R. Liu, A. Agrawal, W.-k. Liao, and A. Choudhary. "Search space preprocessing in solving complex optimization problems." In: *2014 IEEE International Conference on Big Data (Big Data)*. Washington, DC: IEEE, Oct. 2014, pp. 1–5. ISBN: 978-1-4799-5666-1. DOI: 10.1109/BigData.2014.7154118.
- [Mel06] H. Melzner. "Smaller is Better? Maximization of Good Chips per Wafer by Co-Optimization of Yield and Chip Area." en. In: *The 17th Annual SEMI/IEEE ASMC 2006 Conference*. Boston, MA: IEEE, 2006, pp. 372–379. ISBN: 978-1-4244-0254-0. DOI: 10.1109/ASMC.2006.1638786.

- [Mer+19] F. Merchant, A. Devarajan, A. Basu, D. Ashen, B. Yelton, and P. D. Joshi. "High Performance Memory Repair." In: *2019 IEEE International Symposium on Defect and Fault Tolerance in VLSI and Nanotechnology Systems (DFT)*. Noordwijk, Netherlands: IEEE, Oct. 2019, pp. 1–4. ISBN: 978-1-72812-260-1. DOI: 10.1109/DFT.2019.8875490.
- [Ng+20] W.-K. Ng, W.-T. Tam, A. Hin-Cheung Ko, W.-S. Tam, and C.-W. Kok. "A Review of Wafer Packing and New Results." en. In: *IEEE Solid-State Circuits Magazine* 12.4 (2020), pp. 101–108. ISSN: 1943-0582, 1943-0590. DOI: 10.1109/MSSC.2020.3021845.
- [Pri70] J. Price. "A new look at yield of integrated circuits." In: *Proceedings of the IEEE* 58.8 (1970), pp. 1290–1291. ISSN: 0018-9219. DOI: 10.1109/PROC.1970.7911.
- [Ram] N. H. Ramadan. "Redundancy Yield Model for SRAMS." en. In: ().
- [RTB02] E. Rondey, Y. Tellier, and S. Borri. "A silicon-based yield gain evaluation methodology for embedded-SRAMs with different redundancy scenarios." en. In: *Proceedings of the 2002 IEEE International Workshop on Memory Technology, Design and Testing (MTDT2002)*. Isle of Bendor, France: IEEE Comput. Soc, 2002, pp. 57–61. ISBN: 978-0-7695-1617-2. DOI: 10.1109/MTDT.2002.1029764.
- [Seg+99] J. Segal, S. Bakarian, J. Colburn, M. Kumar, C. Hong, and A. Shubat. "Determining redundancy requirements for memory arrays with critical area analysis." en. In: *Records of the 1999 IEEE International Workshop on Memory Technology, Design and Testing*. San Jose, CA, USA: IEEE Comput. Soc, 1999, pp. 48–53. ISBN: 978-0-7695-0259-5. DOI: 10.1109/MTDT.1999.782683.
- [TH15] Y.-P. Tao and W.-P. Hu. "Design of Sense Amplifier in the High Speed SRAM." In: *2015 International Conference on Cyber-Enabled Distributed Computing and Knowledge Discovery*. Xi'an, China: IEEE, Sept. 2015, pp. 384–387. ISBN: 978-1-4673-9200-6. DOI: 10.1109/CyberC.2015.32.
- [Wid+00] D. Widmann, H. Mader, H. Friedrich, W. Heywang, and R. Müller. *Technology of Integrated Circuits*. Vol. 2. Springer Series in ADVANCED MICRO-ELECTRONICS. Berlin, Heidelberg: Springer Berlin Heidelberg, 2000. ISBN: 978-3-642-08547-5. DOI: 10.1007/978-3-662-04160-4.
- [ZS03] Y. Zorian and S. Shoukourian. "Embedded-memory test and repair: infrastructure IP for SoC yield." en. In: *IEEE Design and Test of Computers* 20.3 (May 2003), pp. 58–66. ISSN: 0740-7475. DOI: 10.1109/MDT.2003.1198687.